

SLIP 1]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Write a C program to create random n elements and sort it in ascending order by using insertion sort.

OR

Q1) Write a C program to perform the following operations using static implementation of stack

- Accept n integers from user and push it into the stack.
- Pop the element from the stack and display it (Use isEmpty())

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database(primary keys are underlined).

Room (roomno,room_name,room_type,charges)

Guest(Guestcode, Gname,city)

The relationship is as follows: Room-Guest: one-to-one. room_type can have values as either 'AC' or 'NonAC'.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- List the details of the rooms having charges between 5000 and 10000.
- List the names of the guests in the sorted order by city name.
- List the minimum charges of a room.
- Increase the charges of all AC rooms by 15%.
- List the names of all the NONAC rooms whose charges are more than 10000.

C. Write a query to list the name of the guest to whom the room with highest charges is allotted. [4 marks]

OR

C. Create a view to list the names of all the NonAC rooms which have charges greater than at least one of the 'AC' room.

Q 1 A)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[50],n,i;
```

```
void insertion_sort(int arr[], int n);
```

```
printf("How many elements? ");
scanf("%d", &n);

printf("\nEnter the elements: ");
for(i=0;i<n;i++)
{
    scanf("%d",&arr[i]);
}

insertion_sort(arr, n);
printf("\nSorted elements: ");
for(i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

return 0;
}
```

```
void insertion_sort(int arr[], int n)
{
    int j,i,temp;

    for(i=1;i<n;i++)
    {
        temp=arr[i];
        for(j=i-1;j>=0;j--)
        {
```

```
        if(arr[j]>temp)
        {
            arr[j+1]=arr[j];
        }
        else
        {
            break;
        }
    }
    arr[j+1]=temp;
}
```

Q 2)

A)

```
CREATE TABLE Room (
    roomno INT PRIMARY KEY,
    room_name VARCHAR(50),
    room_type VARCHAR(10) CHECK (room_type IN ('AC', 'NonAC')),
    charges NUMERIC(10,2)
);
```

```
CREATE TABLE Guest (
    guestcode INT PRIMARY KEY,
    gname VARCHAR(50),
    city VARCHAR(50),
    roomno INT,
    FOREIGN KEY (roomno) REFERENCES Room(roomno));
```

```
INSERT INTO Room (roomno,room_name, room_type, charges) VALUES
```

```
(1,'Deluxe', 'AC', 8000),
```

```
(2,'Standard', 'NonAC', 4500),
```

```
(3,'Suite', 'AC', 12000),
```

```
(4,'Economy', 'NonAC', 3500),
```

```
(5,'Premium', 'AC', 10000);
```

```
INSERT INTO Guest (guestcode,gname, city, roomno) VALUES
```

```
(101,'Rahul', 'Mumbai', 1),
```

```
(102,'Neha', 'Delhi', 2),
```

```
(103,'Amit', 'Bangalore', 3),
```

```
(104,'Sanya', 'Chennai', 4),
```

```
(105,'Ravi', 'Kolkata', 5);
```

B) ANY 3

1)

```
SELECT *
```

```
FROM Room
```

```
WHERE charges BETWEEN 5000 AND 10000;
```

2)

```
SELECT gname, city
```

```
FROM Guest
```

```
ORDER BY city;
```

3)

```
SELECT MIN(charges) AS min_charges
```

```
FROM Room;
```

C 1)

SELECT g.gname

FROM Guest g

JOIN Room r ON g.roomno = r.roomno

WHERE r.charges = (SELECT MAX(charges) FROM Room);

SLIP 2]

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q.1) Write a C program to implement static Implementation of Queue of integers with following operation:

- Initialize()
- Insert()
- Delete()
- Iseempty()
- Display()

OR

Q.1) Write a menu driven program for the following operations on singly linked list.

- Create and display a linked list of n nodes.
- Search a particular element from a linked list.

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following database(primary keys are underlined).

Employee(eno,ename,designation,salary)

Department(dno,dname,location)

The relationship is as follows: Employee-Department: **many-to-one**.

- Create the above database in PostGreSQL and insert sufficient records. [5 marks]
- Execute the following queries in PostGreSQL (any 3) [6 marks]
 - Give a 5% raise in salary to all the employees.
 - Display average salary of an employee.
 - List the details of all the departments located at 'Nashik' city.
 - Display the details of employees whose names ends with an alphabet 'a'.
 - Display the location of 'HR' department.

C. Write a query to list the details of employees who do not work in any of the departments located at 'Delhi'. [4 marks]

OR

C. Create a view to list the names of employees whose salary is greater than all the employees working in 'HR' department.

Q 2 A)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#define max 5
```

```
struct queue {  
    int data[max];
```

```
    int rear, front;
};

void initq(struct queue *q)
{
    q->rear = q->front = -1;
}

int isfull(struct queue *q)
{
    return (q->rear == max-1);
}

int isempty(struct queue *q)
{
    return (q->rear == q->front);
}

void addq(struct queue *q,int n)
{
    q->rear++;
    q->data[q->rear]=n;
}

int delq(struct queue *q)
{
    q->front++;
    return (q->data[q->front]);
}
```

```
int peek(struct queue *q)
{
    return (q->data[q->front+1]);
}
```

```
void display(struct queue *q)
{
    int i;
    printf("\nQueue elements are: ");
    for(i=q->front+1; i<=q->rear; i++)
    {
        printf("%d ", q->data[i]);
    }
}
```

```
void main()
{
    int n,ch;
    struct queue q1;

    initq(&q1);

    do
    {
        printf("\n1.Add\n2.Delete\n3.Display\n4.Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&ch);
        switch(ch)
```

```

{
    case 1: if(isfull(&q1))
        printf("\nQueue is full");
        else
        {
            printf("\nEnter the data: ");
            scanf("%d",&n);
            addq(&q1,n);

            display(&q1);
        }
        break;
    case 2: if(isempty(&q1))
        printf("\nQueue is empty");
        else
        {
            printf("\nDeleted element is %d",delq(&q1));
            display(&q1);
        }
        break;
    case 3: if(isempty(&q1))
        printf("\nQueue is empty");
        else
            printf("\nPeak element is %d",peek(&q1));
        break;
    default: printf("\nInvalid choice");
}

} while(ch!=4);

```

```
}
```

Q2 A)

```
CREATE TABLE Department (  
    dno INT PRIMARY KEY,  
    dname VARCHAR(50),  
    location VARCHAR(50)  
);
```

```
CREATE TABLE Employee (  
    eno INT PRIMARY KEY,  
    ename VARCHAR(50),  
    designation VARCHAR(50),  
    salary NUMERIC,  
    dno INT,  
    FOREIGN KEY (dno) REFERENCES Department(dno)  
);
```

```
INSERT INTO Department (dno, dname, location) VALUES  
(1, 'HR', 'Nashik'),  
(2, 'IT', 'Pune'),  
(3, 'Finance', 'Mumbai'),  
(4, 'Admin', 'Nashik');
```

```
INSERT INTO Employee (eno, ename, designation, salary, dno) VALUES  
(101, 'Rahul', 'Manager', 50000, 1),  
(102, 'Neha', 'Executive', 30000, 2),
```

```
(103, 'Amit', 'Analyst', 40000, 3),  
(104, 'Sanya', 'Clerk', 25000, 4),  
(105, 'Ravi', 'Developer', 35000, 2);
```

B)

3)

```
SELECT *  
  
FROM Department  
  
WHERE location = 'Nashik';
```

4)

```
SELECT *  
  
FROM Employee  
  
WHERE ename LIKE '%a';
```

5)

```
SELECT location  
  
FROM Department  
  
WHERE dname = 'HR';
```

C 1)

```
SELECT e.*  
  
FROM Employee e  
  
JOIN Department d ON e.dno = d.dno  
  
WHERE d.location <> 'Delhi';
```

SLIP 3]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Accept the array of n integers from user and sort the array in ascending order by using recursive Quick sort algorithm.

OR

Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following database (primary keys are underlined).

Person (pnumber, pname, birthdate, income)

Area (aname, area_type, pincode)

The relationship is as follows: Person-Area: **many-to-one**. The 'area_type' can have values as either 'urban' or 'rural'.

A. Create the above database in PostgreSQL and insert sufficient records.

[5 marks]

B. Execute the following queries in PostgreSQL (any 3)

[6 marks]

i. List the details of all people whose name starts with the alphabet 'R'.

ii. List the names of all people whose birthday falls in the month of 'July'.

iii. Display the details of people in the sorted order of their income.

iv. Display the count of areas of 'urban' type.

v. Change the pincode of 'kalyaninagar' to 411036.

C. Write a query to list the names of people who live in 'Camp' area and have income less than at least one person who lives in 'kalyaninagar' area.

[4 marks]

OR

C. Create a view to list the details of the person with second maximum income.

Q 3) Viva

[5 Marks]

Q 1 B)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[]={14,15,24,8,5,1,2,21};
```

```
int n=sizeof(arr);
```

```
int target;
```

```
int ans;
```

```
int Linear_search(int arr[],int n,int target);
```

```
printf("Enter the number to find: ");
```

```
scanf("%d",&target);
```

```
ans = Linear_search(arr,n,target);
```

```
if(ans == -1)
```

```
{
```

```
printf("\nThe given number not found in the array\n");
```

```
}
```

```
else
```

```
{
```

```
printf("\n%d Found at position %d\n",target,ans+1);
```

```
}
```

```
return 0;
```

```
}
```

```
int Linear_search(int arr[], int n, int target)
```

```
{
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
if(arr[i] == target)
```

```
{
```

```
return i;
```

```
}
```

```
}  
return -1;  
}
```

Q 2)

A)

```
CREATE TABLE Area (  
    aname VARCHAR(50) PRIMARY KEY,  
    area_type VARCHAR(10) CHECK (area_type IN ('urban','rural')),  
    pincode INT  
);
```

```
CREATE TABLE Person (  
    pnumber INT PRIMARY KEY,  
    pname VARCHAR(50),  
    birthdate DATE,  
    income NUMERIC,  
    aname VARCHAR(50),  
    FOREIGN KEY (aname) REFERENCES Area(aname)  
);
```

```
INSERT INTO Area (aname, area_type, pincode) VALUES  
('Kalyaninagar', 'urban', 411001),  
('Hinjewadi', 'urban', 411057),  
('Paud', 'rural', 412108),  
('Wagholi', 'rural', 412207);
```

```
INSERT INTO Person (pnumber, pname, birthdate, income, aname) VALUES
```

(1, 'Rahul', '1998-07-15', 55000, 'Kalyaninagar'),
(2, 'Neha', '2000-03-20', 45000, 'Hinjewadi'),
(3, 'Ramesh', '1995-07-05', 60000, 'Paud'),
(4, 'Amit', '1992-11-22', 70000, 'Wagholi'),
(5, 'Ravi', '1999-01-10', 48000, 'Hinjewadi');

B)

1)

```
SELECT *  
FROM Person  
WHERE pname LIKE 'R%';
```

3)

```
SELECT *  
FROM Person  
ORDER BY income;
```

5)

```
UPDATE Area  
SET pincode = 411036  
WHERE aname = 'Kalyaninagar';
```

C 2)

```
CREATE OR REPLACE VIEW SecondMaxIncomePerson AS  
SELECT *  
FROM Person  
WHERE income = (  
    SELECT MAX(income)  
    FROM Person
```

```
WHERE income < (SELECT MAX(income) FROM Person)  
);
```

SLIP 4]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Read the data from file 'student.txt' containing names of student and their class (FY, SY, TY). Accept a student name from user and use linear search algorithm to check whether the name is present in the file and output the Class of student otherwise output "Student not in the list".

OR

Q 1) Write a 'C' program to reverse a string using Static implementation of Stack.

Section II: Database Management Systems

[15 marks]

Q.2) Consider the following database (primary keys are underlined).

Policy (pno, pname,premium_amt,policy_type)

Customer (cno,cname.city, agent_name)

The relationship is as follows: Policy-Customer: many-to-one. The 'policy_type' can have values as 'Yearly', 'Half-yearly' or 'Monthly'.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- List the details of all customers who live in city.
- Display the average premium amount.
- Count the number of customers who have taken 'Jeevan Anand' policy.
- Increase the premium amount for 'Monthly' policies by 10%.
- Display the policy_type wise count of policies.

C. Write a query to list the policy_type with highest average premium amount.

[4 marks]

OR

C. Create a view to list the name of the customer who has taken highest number of policies.

Q 1 B)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h> // for strlen

#define sz 10

typedef struct stack {
    int stk[sz];
    int top;
```

```
} STACK;
```

```
void initstack(STACK *s) {  
    s->top = -1;  
}
```

```
int isfull(STACK *s) {  
    return (s->top == sz - 1);  
}
```

```
int isempty(STACK *s) {  
    return (s->top == -1);  
}
```

```
void push(STACK *s, int n) {  
    if (isfull(s)) {  
        printf("\nStack overflow!");  
        return;  
    }  
    s->stk[++s->top] = n;  
}
```

```
int pop(STACK *s) {  
    if (isempty(s)) {  
        printf("\nStack underflow!");  
        return -1;  
    }  
    return s->stk[s->top--];  
}
```

```

int main() {
    char str[30];
    STACK s1;
    int i, len;

    printf("\nEnter the string: ");
    scanf("%s", str);

    len = strlen(str);

    initstack(&s1);

    // Push all characters
    for (i = 0; i < len; i++) {
        push(&s1, str[i]);
    }

    while(!isempty(&s1)) {
        printf("%c", pop(&s1));
    }

    return 0;
}

```

Q 2)

1)

SELECT *

FROM Customer

```
WHERE city = 'Mumbai';
```

2)

```
SELECT AVG(premium_amt) AS average_premium  
FROM Policy;
```

3)

```
SELECT policy_type, COUNT(*) AS policy_count  
FROM Policy  
GROUP BY policy_type;
```

C 1)

```
SELECT policy_type  
FROM Policy  
GROUP BY policy_type  
HAVING AVG(premium_amt) = (  
    SELECT MAX(avg_premium)  
    FROM (  
        SELECT policy_type, AVG(premium_amt) AS avg_premium  
        FROM Policy  
        GROUP BY policy_type  
    ) AS subquery  
);
```

Slip 5]

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Write a C program to implement singly linked list with Create and Display operation.

OR

Q 1) Read the data from file 'student.txt' containing names of student and their class(FY,SY,TY). Accept a student name from user and use binary search algorithm to check whether the name is present in the file and output the class of student otherwise output "Student not in the list".

Section II: Database Management Systems

[15 marks]

Q.2) Consider the following database (primary keys are underlined).

Doctor (dno, dname, city)

Patient (pno, pat_name, city, disease)

The relationship is as follows: Doctor-Patient: **many-to-many** with **Fees** as a descriptive attribute.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- List the details of all the doctors from 'Nagpur' city.
- Display count of patients.
- List the names of the patients suffering from 'cancer' disease.
- Change the city of 'Dr. Patil' to Pune.
- List the names of patients that starts with alphabet 'A'

C. Write a query to find the number of patients suffering from "Asthama" and have been charged a fee of Rs. 200 by "Dr. Kumar".

[4 marks]

OR

C. Create a view to find the name of a doctor who treats maximum number of patients.

Q1 A)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node{  
    int data;  
    struct node *next;  
};
```

```
struct node *create();
```

```
void display(struct node *head);
```

```
int main()
```

```
{
```

```
    struct node *head = NULL, *head1 = NULL, *head2 = NULL;
```

```
    int choice;
```

```
    while(1)
```

```
    {
```

```
        printf("\n1. Create List\n");
```

```
        printf("2. Display List\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch(choice)
```

```
        {
```

```
            case 1: head = create();
```

```
                break;
```

```
            case 2: display(head);
```

```
                break;
```

```
            case 0: exit(0);
```

```
            default: printf("Invalid choice\n");
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
struct node *create()
```

```
{
```

```

int i,n;

struct node *head = NULL, *nw, *curr;

printf("Enter number of nodes: "); // <-- Missing in your code
scanf("%d",&n);

for(i=0;i<n;i++)
{
    nw= (struct node *)malloc(sizeof(struct node));
    nw->next = NULL;
    printf("Enter data: ");
    scanf("%d", &nw->data);
    if(head == NULL)
    {
        head = curr = nw;
    }
    else{
        curr->next = nw;
        curr = nw;
    }
}
return head;
}

```

```

void display(struct node *head)
{
    struct node *curr = head;
    while(curr!=NULL)
    {

```

```
        printf("\t%d", curr->data);  
        curr = curr->next;  
    }  
}
```

Q2)

A)

```
CREATE TABLE Doctor (  
    dno INT PRIMARY KEY,  
    dname VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Patient (  
    pno INT PRIMARY KEY,  
    pat_name VARCHAR(50),  
    city VARCHAR(50),  
    disease VARCHAR(50)  
);
```

```
CREATE TABLE DoctorPatient (  
    dno INT,  
    pno INT,  
    fees NUMERIC(10,2),  
    PRIMARY KEY (dno, pno),  
    FOREIGN KEY (dno) REFERENCES Doctor(dno),  
    FOREIGN KEY (pno) REFERENCES Patient(pno)  
);
```

```
INSERT INTO Doctor(dno,dname,city)
```

```
VALUES
```

```
(1, 'Dr. Patil', 'Nagpur'),
```

```
(2, 'Dr. Sharma', 'Pune'),
```

```
(3, 'Dr. Mehta', 'Nagpur');
```

```
INSERT INTO Patient (pno, pat_name, city, disease) VALUES
```

```
(101, 'Amit', 'Nagpur', 'Fever'),
```

```
(102, 'Anita', 'Pune', 'Cancer'),
```

```
(103, 'Ravi', 'Mumbai', 'Diabetes'),
```

```
(104, 'Ajay', 'Nagpur', 'Cancer');
```

```
INSERT INTO DoctorPatient (dno, pno, fees) VALUES
```

```
(1, 101, 500),
```

```
(1, 102, 1000),
```

```
(2, 103, 800),
```

```
(3, 104, 1200);
```

B)

1)

```
SELECT * FROM Doctor
```

```
WHERE city = 'Nagpur';
```

2)

```
SELECT pat_name
```

```
FROM Patient
```

```
WHERE disease = 'Cancer';
```

3)

```
SELECT pat_name  
FROM Patient  
WHERE pat_name LIKE 'A%';
```

C 1)

```
SELECT COUNT(*) AS patient_count  
FROM Patient p  
JOIN DoctorPatient dp ON p.pno = dp.pno  
JOIN Doctor d ON dp.dno = d.dno  
WHERE p.disease = 'Asthama'  
AND dp.fees = 200  
AND d.dname = 'Dr. Kumar';
```

Slip 6]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

OR

Q 1) Write a menu driven program for the following operations on singly link list.

- Create and display a linked list of n nodes.
- Search an element in a linked list.

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following database(primary keys are underlined).

Student (no,name,city)

Teacher(tno,tname,phone_no,salary)

The relationship is as follows: Student-Teacher: **many-to-many** with **subject** as a descriptive attribute.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- List the names of students from 'Ahilyanagar' city.
- Display the count of teachers whose salary is between 40000 to 60000.
- Change the phone number of 'Prof. Patil' to '9822131226'
- List the details of the teachers in the sorted order of their name.
- List the names of the teachers who have salary less than 50000.

C. Write a query to list the names of all teachers who do not teach any subject that is taught by 'Mr. Patil'. [4 marks]

OR

C. Create a view to list the details of students who have taken 'DBMS' subject.

Q 1 A)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[50],n,i;
```

```
void selection_sort(int arr[], int n);
```

```
printf("How many elements? ");  
scanf("%d", &n);
```

```
printf("\nEnter the elements: ");  
for(i=0;i<n;i++)  
{  
    scanf("%d",&arr[i]);  
}
```

```
selection_sort(arr, n);
```

```
printf("\nSorted elements: ");  
for(i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

```
return 0;  
}
```

```
void selection_sort(int arr[], int n)  
{  
    int i,j;  
    for(i=0;i<n-1;i++)  
    {  
        int min=i;  
        for(j=i+1;j<n;j++)  
        {  
            if(arr[j]<arr[min])
```

```

        {
            min=j;
        }
    }

    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}

printf("\nSorted array: ");
for(i=0;i<n;i++)
{
    printf("\t%d", arr[i]);
}
printf("\n");

}

```

Q 2)

A)

```

CREATE TABLE Student (
    rno INT PRIMARY KEY,
    name VARCHAR(50),
    city VARCHAR(50)
);

```

```

CREATE TABLE Teacher (
    tno INT PRIMARY KEY,
    tname VARCHAR(50),

```

```
    phone_no VARCHAR(15),  
    salary NUMERIC  
);
```

```
CREATE TABLE StudentTeacher (  
    rno INT,  
    tno INT,  
    subject VARCHAR(50),  
    PRIMARY KEY (rno, tno, subject),  
    FOREIGN KEY (rno) REFERENCES Student(rno),  
    FOREIGN KEY (tno) REFERENCES Teacher(tno)  
);
```

```
INSERT INTO Student (rno, name, city) VALUES  
(1, 'Amit', 'Ahilyanagar'),  
(2, 'Neha', 'Pune'),  
(3, 'Ravi', 'Ahilyanagar'),  
(4, 'Sanya', 'Mumbai');
```

```
INSERT INTO Teacher (tno, tname, phone_no, salary) VALUES  
(101, 'Prof. Patil', '9876543210', 55000),  
(102, 'Prof. Sharma', '9123456780', 45000),  
(103, 'Prof. Mehta', '9988776655', 60000);
```

```
INSERT INTO StudentTeacher (rno, tno, subject) VALUES  
(1, 101, 'Math'),  
(2, 101, 'Physics'),  
(3, 102, 'Chemistry'),  
(4, 103, 'Biology'),
```

(1, 103, 'English');

B)

1)

```
SELECT name
FROM Student
WHERE city = 'Ahilyanagar';
```

4)

```
SELECT *
FROM Teacher
ORDER BY tname;
```

5)

```
SELECT tname
FROM Teacher
WHERE salary < 50000;
```

C 2)

```
CREATE OR REPLACE VIEW DBMS_Students AS
SELECT s.*
FROM Student s
JOIN StudentTeacher st ON s.rno = st.rno
WHERE st.subject = 'DBMS';

SELECT * FROM DBMS_Students;
```

Slip 7]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

OR

Q 1) Read the data from "Student.txt (Stud_name, Age, Percentage)" file and sort on Age using insertion sort. (Create a separate sorted file).

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following database(primary keys are underlined).

Item (itemno, name, quantity)

Supplier (sno, name, city)

The relationship is as follows: Item-Supplier: **many-to-many** with **rate** as a descriptive attribute.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- Change the quantity of item 'Mouse' to 80.
- List the details of the suppliers whose name begins with the alphabet 'M'.
- Display the total count of items.
- List the names of suppliers who do not live in Pune city.
- List the names of items with quantity less than 10.

C. Write a query to display the name of supplier who supplies Item with minimum rate.

[4 marks]

OR

C. Create a view to display supplier-wise list of items.

Q 1 A)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[50],n;
```

```
printf("Enter the size of the array");
```

```
scanf("%d",&n);
```

```
printf("\nEnter the sorted array: \n");
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
scanf("%d",&arr[i]);
```

```
}
```

```
int ans,target;
```

```
printf("Enter the number to search: ");
```

```
scanf("%d",&target);
```

```
int Binary_search(int arr[], int n, int target);
```

```
ans= Binary_search(arr,n,target);
```

```
if(ans == -1)
```

```
{
```

```
printf("Not Found");
```

```
}
```

```
else
```

```
{
```

```
printf("Found at position %d",ans+1);
```

```
}
```

```
return 0;
```

```
}
```

```
int Binary_search(int arr[], int n, int target)
```

```
{
```

```
int st=0, end = n-1;

while(st<=end)
{
int mid = (st+end)/2;

if(target == arr[mid])
{
return mid;
}
else if(target<arr[mid])
{
end = mid-1;
}
else
{
st=mid+1;
}

}
return -1;
}
```

Q2)

A)

```
CREATE TABLE Item (
    itemno INT PRIMARY KEY,
    name VARCHAR(50),
    quantity INT
```

);

CREATE TABLE Supplier (

sno INT PRIMARY KEY,

name VARCHAR(50),

city VARCHAR(50)

);

CREATE TABLE ItemSupplier (

itemno INT,

sno INT,

rate NUMERIC,

PRIMARY KEY (itemno, sno),

FOREIGN KEY (itemno) REFERENCES Item(itemno),

FOREIGN KEY (sno) REFERENCES Supplier(sno)

);

INSERT INTO Item (itemno, name, quantity) VALUES

(1, 'Mouse', 50),

(2, 'Keyboard', 30),

(3, 'Monitor', 15),

(4, 'CPU', 5),

(5, 'Printer', 8);

INSERT INTO Supplier (sno, name, city) VALUES

(101, 'Mahesh', 'Mumbai'),

(102, 'Ramesh', 'Pune'),

(103, 'Mona', 'Nagpur'),

(104, 'Suresh', 'Pune'),

(105, 'Anita', 'Delhi');

INSERT INTO ItemSupplier (itemno, sno, rate) VALUES

(1, 101, 500),

(1, 102, 520),

(2, 103, 800),

(3, 104, 15000),

(4, 105, 30000),

(5, 101, 700);

B)

2)

SELECT *

FROM Supplier

WHERE name LIKE 'M%';

3)

SELECT COUNT(*) AS total_items

FROM Item;

5)

SELECT name

FROM Item

WHERE quantity < 10;

C 1)

SELECT s.name

FROM Supplier s

JOIN ItemSupplier isup ON s.sno = isup.sno

```
WHERE isup.rate = (  
    SELECT MIN(rate)  
    FROM ItemSupplier  
);
```

Slip 8]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

OR

Q1) Write a menu driven program for the following operations on doubly link list.

- Create and Display a link list of n nodes
- Insert an element in a linked list in a particular position

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following Student-Teacher database(primary keys are underlined).

Student (sno , s_name, s_class)

Teacher (tno , t_name, yrs_experience)

The relationship is as follows:

Student-Teacher: **Many to Many** with descriptive attribute **Subject**.

Constraint:s_class should be either "FY", "SY" or "TY"

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- Give class-wise number of students.
- List all students studying in class "TY".
- Count the number of students who have taken subject "Maths"
- Delete record of student whose sno "105"
- Add designation column in teacher table

C. Write a query to list the names of all teachers with their subjects along with the total number of students they are teaching. [4 marks]

OR

C. Create a view to find the teacher who teaches maximum number of subjects.

Q1 A)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[50],n;
```

```
void Bubble_sort(int arr[], int n);
```

```
printf("How many elements? ");
```

```
scanf("%d",&n);
```

```
printf("Enter the elements: \n");
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
scanf("%d",&arr[i]);
```

```
}
```

```
Bubble_sort(arr,n);
```

```
return 0;
```

```
}
```

```
void Bubble_sort(int arr[], int n)
```

```
{
```

```
int i,j;
```

```
for(i=0;i<n-1;i++)
```

```
{
```

```
for(j=0;j<n-i-1;j++)
```

```
{
```

```
if(arr[j]>arr[j+1])
```

```
{
```

```
int temp = arr[j];
```

```
arr[j]=arr[j+1];
```

```
arr[j+1]=temp;
```

```
}
```

```
}  
}
```

```
printf("\nSorted array: ");  
for(i=0;i<n;i++)  
{  
printf("%d\t",arr[i]);  
}  
printf("\n");  
}
```

Q 2)

A)

```
CREATE TABLE Student (  
    sno INT PRIMARY KEY,  
    s_name VARCHAR(50),  
    s_class VARCHAR(2) CHECK (s_class IN ('FY', 'SY', 'TY'))  
);
```

```
CREATE TABLE Teacher (  
    tno INT PRIMARY KEY,  
    t_name VARCHAR(50),  
    yrs_experience INT  
);
```

```
CREATE TABLE StudentTeacher (  
    sno INT,  
    tno INT,  
    subject VARCHAR(50),
```

```
PRIMARY KEY (sno, tno, subject),  
FOREIGN KEY (sno) REFERENCES Student(sno),  
FOREIGN KEY (tno) REFERENCES Teacher(tno)  
);
```

```
INSERT INTO Student (sno, s_name, s_class) VALUES  
(101, 'Amit', 'FY'),  
(102, 'Neha', 'SY'),  
(103, 'Ravi', 'TY'),  
(104, 'Sanya', 'TY'),  
(105, 'Anita', 'FY');
```

```
INSERT INTO Teacher (tno, t_name, yrs_experience) VALUES  
(201, 'Prof. Patil', 10),  
(202, 'Prof. Sharma', 8),  
(203, 'Prof. Mehta', 12);
```

```
INSERT INTO StudentTeacher (sno, tno, subject) VALUES  
(101, 201, 'Maths'),  
(102, 201, 'Physics'),  
(103, 202, 'Maths'),  
(104, 203, 'Chemistry'),  
(105, 202, 'Biology');
```

B)

2)

```
SELECT *
```

```
FROM Student
```

```
WHERE s_class = 'TY';
```

4)

```
DELETE FROM Student
```

```
WHERE sno = 105;
```

5)

```
ALTER TABLE Teacher
```

```
ADD COLUMN designation VARCHAR(50);
```

Slip9]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Write a menu driven program for the following operations on singly linked list.

- iii. Create and display a linked list of n nodes.
- iv. Delete a particular element from a linked list.

OR

Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

- i) Accept & Display Function
- ii) Linear Search Function

Section II: Database Management Systems

[15 marks]

Q 2) Create the following tables (primary keys are underlined).

Person (pnumber, pname, birthdate, income),

Area(aname, area_type).

There exists a **one-to-many** relationship between Area and Person.

Constraint: 'area_type' should have value either **urban** or **rural**.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- i. List the details of persons with income greater than 10000.
- ii. Transfer all people living in 'Pune' to 'Delhi'.
- iii. Count number of area's which has type 'urban'.
- iv. Give the count of people who are born in the month of February.
- v. List the names of all people living in 'rural' area.

C. Write a query that count the total number of people staying in "urban" area with income equal to average income. [4 marks]

OR

C. Create a view to print the person's details along with their area name.

Q1 B)

```
#include<stdio.h>
```

```
#include<string.h>
```

```
typedef struct studentfile
```

```
{
```

```
char sname[20];
```

```
char class[20];  
}STUD;  
STUD student[10];
```

```
int read_file()  
{  
    int i=0;  
    FILE *fp;  
    if((fp=fopen("student.txt","r"))!=NULL)  
    {  
        while(!feof(fp))  
        {  
            fscanf(fp,"%s%s",student[i].sname, student[i].class);  
            i++;  
        }  
        fclose(fp);  
    }  
    return (i-1);  
}
```

```
void linear_search(int n, char stud_name[15])  
{  
    int i;  
    for(i=0;i<n;i++)  
    {  
        if(strcmp(student[i].sname, stud_name)==0)  
        {  
            printf("\nstudent class is\n%s\n",student[i].class);  
            break;  
        }  
    }  
}
```

```

}
}
if(i==n)
{
printf("\nstudent name is not in the list\n");
}
}

int main()
{
char student_name[20];
int n,i;
n=read_file(student);
for(i=0;i<n;i++)
    {printf("%s %s\n",student[i].sname, student[i].class);
}
printf("Enter student name: ");
scanf("%s",student_name);
linear_search(n,student_name);

return 0;
}

```

Q2)

A)

```

CREATE TABLE Area (
    aname VARCHAR(50) PRIMARY KEY,
    area_type VARCHAR(10) CHECK (area_type IN ('urban', 'rural'))

```

);

```
CREATE TABLE Person (  
    pnumber INT PRIMARY KEY,  
    pname VARCHAR(50),  
    birthdate DATE,  
    income NUMERIC,  
    aname VARCHAR(50),  
    FOREIGN KEY (aname) REFERENCES Area(aname)  
);
```

```
INSERT INTO Area (aname, area_type) VALUES  
( 'Pune', 'urban'),  
( 'Mumbai', 'urban'),  
( 'Nagpur', 'rural'),  
( 'Kolhapur', 'rural');
```

```
INSERT INTO Person (pnumber, pname, birthdate, income, aname) VALUES  
(1, 'Rahul', '1990-02-15', 12000, 'Pune'),  
(2, 'Neha', '1985-05-20', 9000, 'Mumbai'),  
(3, 'Amit', '1992-02-10', 15000, 'Nagpur'),  
(4, 'Sanya', '1995-08-25', 8000, 'Kolhapur'),  
(5, 'Ravi', '1988-02-28', 20000, 'Pune');
```

B)

1)

```
SELECT *
```

```
FROM Person
```

```
WHERE income > 10000;
```

2)

```
UPDATE Person
```

```
SET aname = 'Delhi'
```

```
WHERE aname = 'Pune';
```

3)

```
SELECT COUNT(*) AS urban_area_count
```

```
FROM Area
```

```
WHERE area_type = 'urban';
```

C 2)

```
CREATE OR REPLACE VIEW PersonAreaDetails AS
```

```
SELECT p.pnumber, p.pname, p.birthdate, p.income, p.aname
```

```
FROM Person p
```

```
JOIN Area a ON p.aname = a.aname;
```

```
SELECT * FROM PersonAreaDetails;
```

Slip 10]

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q1) Create random array of n integers and sort the array in ascending order by using recursive Merge sort algorithm

OR

Q1) Write a C program to implement Static implementation of stack of integers with following operation.

- a. Initialize()
- b. Push()
- c. Pop()
- d. Isempty()
- e. Isfull()
- f. Peek()

Section II: Database Management Systems

[15 marks]

Q 2) Create the following tables (primary keys are underlined).

Account (acctno, acct_type, balance, branch_name)

Customer (custno, cust_name, cust_city)

Relationships: Customer-Account: **One to Many**.

Constraints: acct_type should be “**saving**” or “**current**” and balance is greater than **zero**.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- i. Display information of all saving accounts having balance greater than 5000.
- ii. Count customers in city “Pune”.
- iii. Find the total balance at branch “M.G.Road”.
- iv. Delete the record whose cust_name is “Ramnath”.
- v. Change city of customer “Sanjay” to “Pune”

C. Write a query to find the number of account holders for each city. [4 marks]

OR

C. Create a view to print the account details along with customer and their branch details.

Q 3) Viva

[5 marks]

Q1 B)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#define sz 10
```

```
typedef struct stack
```

```
{
```

```
    int stk[sz];
```

```
    int top;
```

```
}STACK;
```

```
STACK s1;
```

```
void initstack(STACK *s1)
```

```
{  
    s1->top=-1;  
}
```

```
int isfull(STACK *s1)
```

```
{  
    return (s1->top == sz-1);  
}
```

```
int isempty(STACK *s1)
```

```
{  
    return (s1->top == -1);  
}
```

```
void push(STACK *s1, int n)
```

```
{  
    s1->top++;  
    s1->stk[s1->top]=n;  
}
```

```
int pop(STACK *s1)
```

```
{  
    return (s1->stk[s1->top--]);  
}
```

```
int peak(STACK *s1)
{
    return (s1->stk[s1->top]);
}
```

```
int display(STACK *s1)
{
    int i;
    printf("Stack elements are: ");
    for(i=s1->top;i>=0;i--)
    {
        printf("%d ",s1->stk[i]);
    }
}
```

```
int main()
{
    int ch,num;
    initstack(&s1);

    while(1)
    {
        printf("\n1. Push\n2. Pop\n3. Peak\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&ch);

        switch(ch)
        {
            case 1: if(isfull(&s1))
```

```

    {
        printf("\nStack is full");
    }
    else
    {
        printf("\nEnter the element to push: ");
        scanf("%d",&num);
        push(&s1,num);
        display(&s1);
    }
    break;
case 2: if(isempty(&s1))
    {
        printf("\nStack is empty");
    }
    else
    {
        printf("\nPopped element is %d\n",pop(&s1));
        display(&s1);
    }
    break;
case 3: if(isempty(&s1))
    {
        printf("\nStack is empty");
    }
    else
    {
        printf("\nTop element is %d",peak(&s1));
    }

```

```

        break;

    case 4: exit(0);

    default: printf("Invalid choice! Please try again.\n");

    }

}

return 0;

}

```

Q2)

A)

```

CREATE TABLE Customer (
    custno INT PRIMARY KEY,
    cust_name VARCHAR(50),
    cust_city VARCHAR(30)
);

```

```

CREATE TABLE Account (
    acctno INT PRIMARY KEY,
    acct_type CHAR(10) CHECK (acct_type IN ('saving','current')),
    balance NUMERIC CHECK (balance > 0),
    branch_name VARCHAR(50),
    custno INT REFERENCES Customer(custno)
);

```

```

INSERT INTO Customer (custno, cust_name, cust_city) VALUES
(1,'Ramnath','Delhi'),
(2,'Sanjay','Mumbai'),
(3,'Anita','Pune'),

```

(4,'Ravi','Pune');

```
INSERT INTO Account (acctno, acct_type, balance, branch_name, custno) VALUES
(101,'saving',6000,'M.G.Road',1),
(102,'current',8000,'Park.Street',2),
(103,'saving',4500,'M.G.Road',3),
(104,'saving',10000,'M.G.Road',4);
```

B)

1)

```
SELECT * FROM Account WHERE acct_type='saving' AND balance>5000;
```

2)

```
DELETE FROM Customer WHERE cust_name='Ramnath';
```

3)

```
UPDATE Customer SET cust_city='Pune' WHERE cust_name='Sanjay';
```

C 1)

```
SELECT c.cust_city, COUNT(a.acctno) AS num_account_holders
FROM Customer c
JOIN Account a ON c.custno = a.custno
GROUP BY c.cust_city;
```

Slip 11]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Write a 'C' program to reverse a string using Static implementation of Stack.

OR

Q 1) Read the data from the 'employee.txt' file and sort on age using Counting sort and write the sorted data to another file 'sortedemponage.txt'.

Section II: Database Management Systems

[15 marks]

Q 2) Create the following tables (**primary keys are underlined**).

Emp (eno, ename, salary)

Project (pno, pname, budget)

Relationships:

Emp– Project : **Many to Many** with descriptive attribute **total_hours**.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. Find the maximum budget.

ii. Increase the salary of all employees by 10%.

iii. Count the number of projects having duration greater than 100 hrs.

iv. List all employees whose name ends with "a".

v. Add column contact_number to employee table.

C. Write a query to list the names of employees who are not working on any project.

[4 marks]

OR

C. Create a view to list the project details having maximum number of hours.

Q 3) Viva

[5 marks]

Q1 A)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h> // for strlen
```

```
#define sz 10
```

```
typedef struct stack {
```

```
    int stk[sz];
```

```
    int top;
```

```
} STACK;
```

```
void initstack(STACK *s) {  
    s->top = -1;  
}
```

```
int isfull(STACK *s) {  
    return (s->top == sz - 1);  
}
```

```
int isempty(STACK *s) {  
    return (s->top == -1);  
}
```

```
void push(STACK *s, int n) {  
    if (isfull(s)) {  
        printf("\nStack overflow!");  
        return;  
    }  
    s->stk[++s->top] = n;  
}
```

```
int pop(STACK *s) {  
    if (isempty(s)) {  
        printf("\nStack underflow!");  
        return -1;  
    }  
    return s->stk[s->top--];  
}
```

```

int main() {
    char str[30];
    STACK s1;
    int i, len;

    printf("\nEnter the string: ");
    scanf("%s", str);

    len = strlen(str);

    initstack(&s1);

    // Push all characters
    for (i = 0; i < len; i++) {
        push(&s1, str[i]);
    }

    while(!isempty(&s1)) {
        printf("%c", pop(&s1));
    }

    return 0;
}

```

Q2)

A)

```

CREATE TABLE Emp (
    eno INT PRIMARY KEY,
    ename VARCHAR(50),

```

```
    salary NUMERIC  
);
```

```
CREATE TABLE Project (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(50),  
    budget NUMERIC  
);
```

```
CREATE TABLE Emp_Project (  
    eno INT REFERENCES Emp(eno),  
    pno INT REFERENCES Project(pno),  
    total_hours INT,  
    PRIMARY KEY (eno, pno)  
);
```

```
INSERT INTO Emp (eno, ename, salary) VALUES  
(1,'Rama',40000),  
(2,'Sita',35000),  
(3,'Kiran',30000),  
(4,'Anita',45000);
```

```
INSERT INTO Project (pno, pname, budget) VALUES  
(101,'Alpha',500000),  
(102,'Beta',300000),  
(103,'Gamma',700000);
```

```
INSERT INTO Emp_Project (eno, pno, total_hours) VALUES  
(1,101,120),
```

(2,101,80),
(3,102,150),
(4,103,90);

B)

1)

SELECT MAX(budget) FROM Project;

2)

UPDATE Emp SET salary = salary * 1.10;

3)

SELECT * FROM Emp

WHERE ename LIKE '%a';

C 1)

SELECT e.ename

FROM Emp e

WHERE e.eno NOT IN (

SELECT ep.eno FROM Emp_Project ep

);

Slip 12]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Perform the following operations using dynamic implementation of stack

- Accept n integers from user and push it into the stack.
- Reverses a string of characters using stack and check whether given string is palindrome or not.

OR

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

Section II: Database Management Systems

[15 marks]

Q 2) Create the following tables (**primary keys are underlined**).

Sales_order(sorderno, s_order_date, order_amt)

Client(clientno, name, address)

There exists a **one-to-many** relationship between Client and Sales_order

Constraints: order_amt should be greater than zero.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- Update the client address of all clients from "Nagpur" to "Kolhapur".
- Delete all sales order whose client number is 30;
- Display all sale records having order date before "23/02/2024".
- Display sales order having maximum order amount.
- Add column order_status to the Sales_order table.

C. Write a query to Display client details having maximum sales orders. [4 marks]

OR

C. Create a view to list names of clients having more than 2 sales orders.

Q 1 B)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[50],n,i;
```

```
void selection_sort(int arr[], int n);
```

```
printf("How many elements? ");
```

```
scanf("%d", &n);
```

```

printf("\nEnter the elements: ");
for(i=0;i<n;i++)
{
    scanf("%d",&arr[i]);
}

selection_sort(arr, n);

printf("\nSorted elements: ");
for(i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

return 0;
}

```

```

void selection_sort(int arr[], int n)
{
    int i,j;
    for(i=0;i<n-1;i++)
    {
        int min=i;
        for(j=i+1;j<n;j++)
        {
            if(arr[j]<arr[min])
            {
                min=j;
            }
        }
    }
}

```

```

        }
    }
    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}

printf("\nSorted array: ");
for(i=0;i<n;i++)
{
    printf("\t%d", arr[i]);
}
printf("\n");
}

```

Q 2)

A)

```

CREATE TABLE Client (
    clientno INT PRIMARY KEY,
    name VARCHAR(50),
    address VARCHAR(50)
);

```

```

CREATE TABLE Sales_order (
    sorderno INT PRIMARY KEY,
    s_order_date DATE,
    order_amt NUMERIC CHECK (order_amt > 0),

```

```
clientno INT REFERENCES Client(clientno)
);
```

```
INSERT INTO Client (clientno, name, address) VALUES
(10,'Ramesh','Nagpur'),
(20,'Suresh','Mumbai'),
(30,'Anita','Nagpur'),
(40,'Vijay','Delhi');
```

```
INSERT INTO Sales_order (sorderno, s_order_date, order_amt, clientno) VALUES
(101,'2024-02-20',5000,10),
(102,'2024-02-25',8000,20),
(103,'2024-01-15',12000,30),
(104,'2024-03-01',7000,40);
```

B)

1)

```
UPDATE Client SET address='Kolhapur' WHERE address='Nagpur';
```

2)

```
DELETE FROM Sales_order WHERE clientno=30;
```

3)

```
SELECT * FROM Sales_order WHERE s_order_date < '2024-02-23';
```

C 2)

```
CREATE VIEW Clients_With_More_Than_2_Orders AS
```

```
SELECT c.clientno, c.name, c.address, COUNT(s.sorderno) AS total_orders
```

```
FROM Client c
```

```
JOIN Sales_order s ON c.clientno = s.clientno  
GROUP BY c.clientno, c.name, c.address  
HAVING COUNT(s.sorderno) > 2;
```

Slip 13]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

OR

Q 1) Perform the following operations using dynamic implementation of Queue

- Insert n elements in a queue.
- Delete the element from the queue and display it.

Section II: Database Management Systems

[15 marks]

Q 2) Create the following tables (**primary keys are underlined**).

Bus (Busno , capacity ,depot_name)

Route (RouteNo ,source ,destination ,no_of_stations)

Relationship: Bus-Route : **Many to one**

Constraint: Bus capacity should not be null.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- List all buses at depot "Swargate".
- Delete Bus whose Bus number is "204".
- List all buses on route number 41.
- List the route details having number of stations greater than 10.
- List all routes starting from "Pune station".

C. Write a query to delete all the buses on routes in which total stations are less than 3.

[4 marks]

OR

C. Create a view to list all buses and route details from "Pimpri" to "Hadapsar".

Q1 B)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct queue
```

```
{
```

```
    int data;
```

```
    struct queue *next;
```

```
}*front,*rear;
```

```
void initq()
```

```
{
```

```
    front=rear=NULL;
```

```
}
```

```
void addq(int n)
```

```
{
```

```
    struct queue *nw;
```

```
    nw=(struct queue *)malloc(sizeof(struct queue));
```

```
    nw->next=NULL;
```

```
    nw->data=n;
```

```
    if(rear==NULL)
```

```
    {
```

```
        front = rear=nw;
```

```
    }
```

```
    else{
```

```
        rear->next=nw;
```

```
        rear=nw;
```

```
    }
```

```
}
```

```
int delq()
```

```
{
```

```
    struct queue *temp=front;
```

```
    int num;
```

```
    num=front->data;
```

```
    front=front->next;
    free(temp);
    if(front==NULL)
        rear=NULL;
    return num;
}
```

```
void display()
{
    struct queue *curr;

    for(curr=front;curr!=NULL;curr=curr->next)
    {
        printf("\t%d ",curr->data);
    }
}
```

```
void main()
{
    int n,ch;
    struct queue q1;

    initq(q1);

    do
    {
        printf("\n1.Add\n2.Delete\n3.Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&ch);
```

```

switch(ch)
{
    case 1:  printf("\nEnter the data: ");
             scanf("%d",&n);
             addq(n);
             display(q1);

             break;
    case 2: if(isempty(q1))
             printf("\nQueue is empty");
            else
            {
                printf("\nDeleted element is %d",delq(q1));
                display(q1);
            }
            break;

    default: printf("\nInvalid choice");
}

} while(ch!=3);
}

```

Q2)

A)

```

CREATE TABLE Route (
    routeno INT PRIMARY KEY,
    source VARCHAR(50),
    destination VARCHAR(50),

```

```
no_of_stations INT  
);
```

```
CREATE TABLE Bus (  
    busno INT PRIMARY KEY,  
    capacity INT NOT NULL,  
    depot_name VARCHAR(50),  
    routeno INT REFERENCES Route(routeno)  
);
```

```
INSERT INTO Route (routeno, source, destination, no_of_stations) VALUES  
(40,'Pune Station','Hadapsar',12),  
(41,'Pune Station','Swargate',8),  
(42,'Swargate','Kothrud',15);
```

```
INSERT INTO Bus (busno, capacity, depot_name, routeno) VALUES  
(201,50,'Swargate',40),  
(202,45,'Kothrud',41),  
(203,60,'Swargate',41),  
(204,55,'Hadapsar',42);
```

B)

1)

```
SELECT * FROM Bus WHERE depot_name='Swargate';
```

2)

```
DELETE FROM Bus WHERE busno=204;
```

3)

```
SELECT * FROM Bus WHERE routeno=41;
```

C 1)

```
DELETE FROM Bus
```

```
WHERE routeno IN (
```

```
    SELECT routeno
```

```
    FROM Route
```

```
    WHERE no_of_stations < 3
```

```
);
```

Slip 14]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Read the data from “Student.txt (Stud_name, Age, Percentage)” file and sort on Percentage using selection sort. (Create a separate sorted file).

OR

Q 1) Write a menu driven program for the following operations on singly linked list.

- Create a linked list and display it.
- Accept an element from user and concatenate it with a created link list.

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following database (**Primary keys are underlined**)

Doctor (dcode, doctor_name, specialization, address, phone_no, fees)

Patient (pcode, Patient_name, symptoms)

There exists a **one-to-many** relationship between Doctor and Patient.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- Find the names of all Patients which start with “M”.
- Count the number of doctors who are Neurologists.
- Give the list of all patients who are suffering from “Fever”
- Find the specialization and phone numbers of all doctors from ‘Sadashiv Peth’.
- Change the address of Dr. Patil to “Camp”.

C. Write a query to display the list of all patients who are treated by doctors staying in ‘Shivajinagar’ area with the specialization “Orthopedic”. [4 marks]

OR

C. Create a view to find the names of patients who are treated by doctor with minimum fees.

Q1 B)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node* create();
```

```
void display(struct node *head);  
struct node* concatenate(struct node *head, int value);
```

```
int main() {  
    struct node *head = NULL;  
    int choice, value;  
  
    while(1) {  
        printf("\nMenu:\n");  
        printf("1. Create and display linked list\n");  
        printf("2. Concatenate an element to the list\n");  
        printf("3. Display list\n");  
        printf("0. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
  
        switch(choice) {  
            case 1:  
                head = create();  
                printf("Created Linked List: ");  
                display(head);  
                break;  
            case 2:  
                printf("Enter element to concatenate: ");  
                scanf("%d", &value);  
                head = concatenate(head, value);  
                printf("After concatenation: ");  
                display(head);  
                break;
```

```

        case 3:
            printf("Linked List: ");
            display(head);
            break;
        case 0:
            exit(0);
        default:
            printf("Invalid choice\n");
    }
}
return 0;
}

```

```

struct node* create() {
    int n, i;

    struct node *head = NULL, *curr, *nw;
    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        nw = (struct node*)malloc(sizeof(struct node));
        nw->next = NULL;
        printf("Enter data: ");
        scanf("%d", &nw->data);

        if(head == NULL)
            head = curr = nw;
        else {
            curr->next = nw;

```

```

        curr = nw;
    }
}
return head;
}

```

```

void display(struct node *head) {
    struct node *curr = head;
    while(curr != NULL) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

```

```

struct node* concatenate(struct node *head, int value) {
    struct node *nw = (struct node*)malloc(sizeof(struct node));
    nw->data = value;
    nw->next = NULL;

    if(head == NULL)
        return nw;

    struct node *curr = head;
    while(curr->next != NULL)
        curr = curr->next;

    curr->next = nw;
    return head;
}

```

```
}
```

```
Q2)
```

```
A)
```

```
CREATE TABLE Doctor (  
    dcode INT PRIMARY KEY,  
    doctor_name VARCHAR(50),  
    specialization VARCHAR(50),  
    address VARCHAR(50),  
    phone_no VARCHAR(15),  
    fees NUMERIC  
);
```

```
CREATE TABLE Patient (  
    pcode INT PRIMARY KEY,  
    patient_name VARCHAR(50),  
    symptoms VARCHAR(50),  
    dcode INT REFERENCES Doctor(dcode)  
);
```

```
INSERT INTO Doctor (dcode, doctor_name, specialization, address, phone_no, fees) VALUES  
(1,'Dr. Patil','Neurologist','Sadashiv Peth','9876543210',500),  
(2,'Dr. Mehta','Cardiologist','Camp','9123456780',600),  
(3,'Dr. Joshi','Dermatologist','Kothrud','9988776655',400);
```

```
INSERT INTO Patient (pcode, patient_name, symptoms, dcode) VALUES  
(101,'Maya','Fever',1),  
(102,'Manish','Cold',1),  
(103,'Anita','Fever',2),  
(104,'Rohan','Cough',3);
```

B)

1)

```
SELECT patient_name FROM Patient WHERE patient_name LIKE 'M%';
```

2)

```
SELECT * FROM Patient WHERE symptoms='Fever';
```

3)

```
SELECT specialization, phone_no FROM Doctor WHERE address='Sadashiv Peth';
```

C 1)

```
SELECT p.patient_name, p.symptoms
```

```
FROM Patient p
```

```
JOIN Doctor d ON p.dcode = d.dcode
```

```
WHERE d.address='Shivajinagar' AND d.specialization='Orthopedic';
```

Slip 15]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using insertion sort algorithm.

OR

Q 1) Perform the following operations using dynamic implementation of Queue
i. Insert n elements in a queue.
ii. Delete the element from the queue and display it.

Section II: Database Management Systems

[15 marks]

Q 2) Create the following tables (**primary keys are underlined**).

Game (gname, noofplayers, coachname, captain_name)

Player (pno, pname)

There exists a **one-to-many** relationship between Game and Player.

Constraint **captain_name** should be **uppercase**.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- List the names of players playing "hockey".
- Give the average number of players.
- Delete the records of players playing 'kho kho'.
- List names of players not playing "cricket".
- Update the coach name from "Dhoni" to "Tendulkar" for game "Cricket".

C. Write a query to display the game details with maximum number of players. [4 marks]

OR

C. Create a view to print game-wise list of Players along with their game name

Q1 A)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[50],n,i;
```

```
void insertion_sort(int arr[], int n);
```

```
printf("How many elements? ");
```

```
scanf("%d", &n);
```

```

printf("\nEnter the elements: ");
for(i=0;i<n;i++)
{
    scanf("%d",&arr[i]);
}

insertion_sort(arr, n);

printf("\nSorted elements: ");
for(i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

return 0;
}

```

```

void insertion_sort(int arr[], int n)
{
    int j,i,temp;

    for(i=1;i<n;i++)
    {
        temp=arr[i];
        for(j=i-1;j>=0;j--)
        {
            if(arr[j]>temp)
            {
                arr[j+1]=arr[j];

```

```

    }
    else
    {
        break;
    }
}

arr[j+1]=temp;
}
}

```

Q2)

A)

```

CREATE TABLE Game (
    gname VARCHAR(50) PRIMARY KEY,
    noofplayers INT,
    coachname VARCHAR(50),
    captain_name VARCHAR(50) CHECK (captain_name = UPPER(captain_name))
);

```

```

CREATE TABLE Player (
    pno INT PRIMARY KEY,
    pname VARCHAR(50),
    gname VARCHAR(50) REFERENCES Game(gname)
);

```

```

INSERT INTO Game (gname, noofplayers, coachname, captain_name) VALUES
('Hockey', 11, 'Dhoni', 'RAHUL'),
('Cricket', 11, 'Dhoni', 'VIRAT'),

```

('Kho Kho', 12, 'Kumar', 'RAJESH');

INSERT INTO Player (pno, pname, gname) VALUES

(1,'Rahul','Hockey'),

(2,'Vikas','Hockey'),

(3,'Virat','Cricket'),

(4,'Sachin','Cricket'),

(5,'Rajesh','Kho Kho'),

(6,'Amit','Kho Kho');

B)

1)

SELECT pname FROM Player WHERE gname='Hockey';

3)

DELETE FROM Player WHERE gname='Kho Kho';

4)

SELECT pname FROM Player WHERE gname<>'Cricket';

C 1)

SELECT * FROM Game

WHERE noofplayers = (SELECT MAX(noofplayers) FROM Game);

Slip 17)

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using recursive Quick sort algorithm.

OR

Q 1) Perform the following operations using implementation of stack (Static or dynamic)

- Accept n integers from user and push it into the stack.
- Reverses a string of characters.

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following database (**Primary keys are underlined**)

Property (Pno, area_Sqft, location, city)

Owner (oname, address, phone)

The relationship is as follows:

Property – Owner are related with **Many to One** relationship.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- Give the list of all properties in Nashik city.
- Give the names of owners whose property is greater than 2000sq feet.
- Find the number of properties owned by “Mr.Rahane”.
- Count all properties of “Pune” city.
- Delete all properties in “Pune” city owned by “Mr.Baravkar”.

C. Write a query to find the names of all owners who own a property in Pune city, located in ‘Kothrud’ area and which has an area greater than 10000sqft.

OR

C. Create a view to find the owner wise list of properties. [4 marks]

Slip 4 Q1 B)

Q2)

A)

CREATE TABLE Owner (

oname VARCHAR(50) PRIMARY KEY,

address VARCHAR(50),

phone VARCHAR(15)

);

```
CREATE TABLE Property (  
    pno INT PRIMARY KEY,  
    area_sqft NUMERIC,  
    location VARCHAR(50),  
    city VARCHAR(50),  
    oname VARCHAR(50) REFERENCES Owner(oname)  
);
```

```
INSERT INTO Owner (oname, address, phone) VALUES  
( 'Mr.Rahane', 'Pune', '9876543210'),  
( 'Mr.Baravkar', 'Mumbai', '9123456780'),  
( 'Mr.Sharma', 'Nashik', '9988776655');
```

```
INSERT INTO Property (pno, area_sqft, location, city, oname) VALUES  
(101,1500,'Kothrud','Pune','Mr.Rahane'),  
(102,2500,'Deccan','Pune','Mr.Rahane'),  
(103,3000,'Swargate','Pune','Mr.Baravkar'),  
(104,1200,'Gangapur','Nashik','Mr.Sharma'),  
(105,1800,'Nashik City','Nashik','Mr.Sharma');
```

B)

1)

```
SELECT * FROM Property WHERE city='Nashik';
```

3)

```
SELECT COUNT(*) FROM Property WHERE oname='Mr.Rahane';
```

4)

```
SELECT COUNT(*) FROM Property WHERE city='Pune';
```

C 1)

```
SELECT DISTINCT oname FROM Property
```

```
WHERE city='Pune' AND location='Kothrud' AND area_sqft > 10000;
```

Slip 18]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

OR

Q1) Write a program to perform the following operation on stack.

Convert an infix expression of the form $(a*(b-c*d)/((a+d)/b))$ into its equivalent postfix notation.

Section II: Database Management Systems

[15 marks]

Q 2) Consider the following database(**Primary keys are underlined**)

Student (rno,name,city)

Teacher(tno,tname,phone_no,salary)

The relationship is as follows:

Student-Teacher: **Many-to-Many** with **subject** as a descriptive attribute.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. List the students from "Sangamner"city.

ii. Find the maximum salary of teachers.

iii. Change the phone number of teacher 'Dr. Mulay' to 9834233235.

iv. List the details of the teachers in the sorted order of their name.

v. Count the number of students who have taken subject "IKS".

C. Write a query to list the names of all students who are taught by teacher "Dr.Wani"

[4 marks]

OR

C. Create a view to list the details of students who have not taken 'Electronics'subject.

Q 1 A)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[50],n,i;
```

```
void bubble_sort(int arr[], int n);
```

```
printf("How many elements? ");
```

```
scanf("%d", &n);
```

```
printf("\nEnter the elements: ");  
for(i=0;i<n;i++)  
{  
    scanf("%d",&arr[i]);  
}  
  
bubble_sort(arr, n);  
  
return 0;  
}
```

```
void bubble_sort(int arr[], int n)  
{  
    int i,j;  
  
    for(i=0;i<n-1;i++)  
    {  
        for(j=0;j<n-i;j++)  
        {  
            if(arr[j]>arr[j+1])  
            {  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
            }  
        }  
    }  
}
```

```
printf("\nSorted array: ");  
for(i=0;i<n;i++)  
{  
    printf("\t%d", arr[i]);  
}  
printf("\n");  
}
```

Q 2)

A)

```
CREATE TABLE Student (  
    rno INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Teacher (  
    tno INT PRIMARY KEY,  
    tname VARCHAR(50),  
    phone_no VARCHAR(15),  
    salary NUMERIC  
);
```

```
CREATE TABLE Student_Teacher (  
    rno INT REFERENCES Student(rno),  
    tno INT REFERENCES Teacher(tno),  
    subject VARCHAR(50),  
    PRIMARY KEY (rno, tno, subject)  
);
```

-- Insert sample records

INSERT INTO Student (rno, name, city) VALUES

(1,'Amit','Sangamner'),

(2,'Rohit','Pune'),

(3,'Neha','Sangamner'),

(4,'Sneha','Nashik');

INSERT INTO Teacher (tno, tname, phone_no, salary) VALUES

(101,'Dr.Mulay','9876543210',50000),

(102,'Dr.Wani','9123456780',60000),

(103,'Dr.Patil','9988776655',45000);

INSERT INTO Student_Teacher (rno, tno, subject) VALUES

(1,101,'IKS'),

(2,101,'IKS'),

(3,102,'Electronics'),

(4,103,'Maths'),

(2,102,'IKS');

B)

1) SELECT * FROM Student WHERE city='Sangamner';

2) SELECT MAX(salary) AS max_salary FROM Teacher;

3) SELECT * FROM Teacher ORDER BY tname;

C both)

SELECT s.name

```
FROM Student s
JOIN Student_Teacher st ON s.rno = st.rno
JOIN Teacher t ON st.tno = t.tno
WHERE t.tname='Dr.Wani';
```

-- OR

```
CREATE VIEW Students_Not_Electronics AS
SELECT *
FROM Student
WHERE rno NOT IN (
    SELECT rno FROM Student_Teacher WHERE subject='Electronics'
);
```

Slip 20]

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q1) Read the data from the 'employee.txt' file and sort on age using Merge sort. Write the sorted data to another file 'sortedemponage.txt'.

OR

Q1) Write a C program to implement Static implementation of stack of integers with following operation.

- a. Initialize()
- b. Push()
- c. Pop()
- d. Iseempty()
- e. Isfull()
- f. Peek()

Section II : Database Management Systems

[15 marks]

Q 2) Consider the following database (Primary keys are underlined)

Customer (cust_no, cust_name, cust_city)

Account (acc_no, acc_type, balance)

Account type can be either "Saving" or "Current"

There exists a **One-to-Many** relationship between Customer and Account.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B .Execute the following queries in PostGreSQL (any 3) [6 marks]

- i. Find the names of all Customers staying in "Delhi".
- ii. Give the total balance of all Saving accounts.
- iii. Find the maximum balance of an account.
- iv. Increase the balance of account no 101123 by Rs 10,000/-
- v. Add column "AADHAR number" to customer table.

C. Write a query to find the names and phone numbers of all Customers of Current accounts, having balance greater than 10,00,000/-

OR

C. Create a View to list details of all customers having saving account and not staying in city "Ahilyanagar".

Q 1 B)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#define sz 10
```

```
typedef struct stack
```

```
{
```

```
int stk[sz];
```

```
    int top;  
}STACK;  
STACK s1;
```

```
void initstack(STACK *s1)  
{  
    s1->top=-1;  
}
```

```
int isfull(STACK *s1)  
{  
    return (s1->top == sz-1);  
}
```

```
int isempty(STACK *s1)  
{  
    return (s1->top == -1);  
}
```

```
void push(STACK *s1, int n)  
{  
    s1->top++;  
    s1->stk[s1->top]=n;  
}
```

```
int pop(STACK *s1)  
{  
    return (s1->stk[s1->top--]);  
}
```

```
int peak(STACK *s1)
{
    return (s1->stk[s1->top]);
}
```

```
int display(STACK *s1)
{
    int i;
    printf("Stack elements are: ");
    for(i=s1->top;i>=0;i--)
    {
        printf("%d ",s1->stk[i]);
    }
}
```

```
int main()
{
    int ch,num;
    initstack(&s1);

    while(1)
    {
        printf("\n1. Push\n2. Pop\n3. Peak\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&ch);

        switch(ch)
        {
```

case 1: if(isfull(&s1))

```
{  
    printf("\nStack is full");  
}  
else  
{  
    printf("\nEnter the element to push: ");  
    scanf("%d",&num);  
    push(&s1,num);  
    display(&s1);  
}  
break;
```

case 2: if(isempty(&s1))

```
{  
    printf("\nStack is empty");  
}  
else  
{  
    printf("\nPopped element is %d\n",pop(&s1));  
    display(&s1);  
}  
break;
```

case 3: if(isempty(&s1))

```
{  
    printf("\nStack is empty");  
}  
else  
{  
    printf("\nTop element is %d",peak(&s1));  
}
```

```

        }
        break;
    case 4: exit(0);
    default: printf("Invalid choice! Please try again.\n");
    }
}

return 0;
}

```

Q2)

A)

```

CREATE TABLE Customer (
    cust_no INT PRIMARY KEY,
    cust_name VARCHAR(50),
    cust_city VARCHAR(50)
);

```

```

CREATE TABLE Account (
    acc_no INT PRIMARY KEY,
    acc_type VARCHAR(10) CHECK (acc_type IN ('Saving','Current')),
    balance NUMERIC CHECK (balance > 0),
    cust_no INT REFERENCES Customer(cust_no)
);

```

```

INSERT INTO Customer (cust_no, cust_name, cust_city) VALUES
(1,'Amit','Delhi'),

```

```
(2,'Rohit','Mumbai'),  
(3,'Neha','Delhi'),  
(4,'Sneha','Ahilyanagar');
```

```
INSERT INTO Account (acc_no, acc_type, balance, cust_no) VALUES  
(101123,'Saving',50000,1),  
(101124,'Current',1500000,1),  
(101125,'Saving',250000,2),  
(101126,'Current',2000000,3),  
(101127,'Saving',30000,4);
```

B)

1)

```
SELECT cust_name FROM Customer WHERE cust_city='Delhi';
```

3)

```
SELECT MAX(balance) AS max_balance FROM Account;
```

5)

```
ALTER TABLE Customer ADD COLUMN aadhar_no VARCHAR(12);
```

C 1)

```
SELECT c.cust_name  
FROM Customer c  
JOIN Account a ON c.cust_no = a.cust_no  
WHERE a.acc_type='Current' AND a.balance > 1000000;
```

Slip 21]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

- Q1) Perform the following operations using static implementation of Queue
- Create a queue of n integers.
 - Delete the element from the queue and display updated queue.

OR

- Q1) Write a menu driven program for the following operations on singly linked list
- Insert an element in a linked list in a particular position.
 - Search an element in a linked list.

Section II : Database Management Systems

[15 marks]

- Q 2) Consider the following database (**Primary keys are underlined**)

Bus (hno, capacity, depot_name)

Driver (dno, driver_name, license_no, address, age, salary)

The relationship is as follows:

Bus and Driver are related with **Many to Many** relationships with the descriptive attribute **Date_of_duty**.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- Find the number of buses having capacity more than 20.
- Count number of drivers who did duty on "15/01/2017"
- Display the license number of all drivers whose name starts with 'S'
- Calculate the average salary of a driver.
- Find all details of driver named "Swapnil"

C. Write a query to find the names of all drivers who were driving buses from "Kothrud Depot" on 12/2/2009.

[4 marks]

OR

C. Create a view to find all details of buses which are driven by drivers whose age is less than 25 years.

Q 1 A)

```
#include <stdio.h>
```

```
#define MAX 100
```

```
struct queue {
```

```
    int data[MAX];
```

```
    int front, rear;
```

```
};
```

// Initialize the queue

```
void initQueue(struct queue *q) {  
    q->front = q->rear = -1;  
}
```

// Check if queue is empty

```
int isEmpty(struct queue *q) {  
    return q->rear == q->front;  
}
```

// Check if queue is full

```
int isFull(struct queue *q) {  
    return q->rear == MAX - 1;  
}
```

// Insert element into queue

```
void enqueue(struct queue *q, int value) {  
    if(isFull(q)) {  
        printf("Queue is full\n");  
        return;  
    }  
    q->rear++;  
    q->data[q->rear] = value;  
}
```

// Delete element from queue

```
int dequeue(struct queue *q) {  
    if(isEmpty(q)) {
```

```

        printf("Queue is empty\n");
        return -1;
    }
    q->front++;
    return q->data[q->front];
}

// Display queue elements
void display(struct queue *q) {
    if(isEmpty(q)) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for(int i = q->front + 1; i <= q->rear; i++)
        printf("%d ", q->data[i]);
    printf("\n");
}

int main() {
    struct queue q;
    int n, value;

    initQueue(&q);

    // i. Create a queue of n integers
    printf("Enter number of elements to insert: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++) {

```

```

        printf("Enter element %d: ", i+1);
        scanf("%d", &value);
        enqueue(&q, value);
    }

    printf("\nQueue after insertion:\n");
    display(&q);

    // ii. Delete the element from the queue and display updated queue
    value = dequeue(&q);
    if(value != -1)
        printf("\nDeleted element: %d\n", value);

    printf("\nQueue after deletion:\n");
    display(&q);

    return 0;
}

```

Q2)

A)

```

CREATE TABLE Bus (
    bno INT PRIMARY KEY,
    capacity INT,
    depot_name VARCHAR(50)
);

```

```

CREATE TABLE Driver (
    dno INT PRIMARY KEY,

```

```
driver_name VARCHAR(50),  
license_no VARCHAR(20),  
address VARCHAR(100),  
age INT,  
salary NUMERIC  
);
```

```
CREATE TABLE Bus_Driver (  
    bno INT REFERENCES Bus(bno),  
    dno INT REFERENCES Driver(dno),  
    date_of_duty DATE,  
    PRIMARY KEY (bno, dno, date_of_duty)  
);
```

```
INSERT INTO Bus (bno, capacity, depot_name) VALUES  
(101, 25, 'Kothrud Depot'),  
(102, 15, 'Swargate Depot'),  
(103, 30, 'Kothrud Depot');
```

```
INSERT INTO Driver (dno, driver_name, license_no, address, age, salary) VALUES  
(1, 'Swapnil', 'LIC123', 'Pune', 24, 30000),  
(2, 'Sanket', 'LIC124', 'Mumbai', 28, 35000),  
(3, 'Rahul', 'LIC125', 'Pune', 23, 28000);
```

```
INSERT INTO Bus_Driver (bno, dno, date_of_duty) VALUES  
(101, 1, '2009-02-12'),  
(102, 2, '2017-01-15'),  
(103, 1, '2009-02-12'),
```

(101, 3, '2017-01-15');

B)

3)

SELECT license_no FROM Driver WHERE driver_name LIKE 'S%';

4)

SELECT AVG(salary) AS avg_salary FROM Driver;

5)

SELECT * FROM Driver WHERE driver_name='Swapnil';

C 1)

SELECT DISTINCT d.driver_name

FROM Driver d

JOIN Bus_Driver bd ON d.dno = bd.dno

JOIN Bus b ON bd.bno = b.bno

WHERE b.depot_name = 'Kothrud Depot' AND bd.date_of_duty = '2009-02-12';

Slip 23]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q1) Write a menu driven program for the following operations on doubly linked list.

- Create and display linked list.
- Reverse a linked list and display it.

OR

Q1) Perform the following operations using static implementation of stack

- Accept n integers from user and push it into the stack.
- Pop the element from the stack and display it (Use isEmpty())

Section II : Database Management Systems

[15 marks]

Q 2) Consider the following database (**Primary keys are underlined**)

Customer (cno, cust_name , cust_city)

Account (ano,acc_type , balance)

There exists a **One to Many** relationships between Customer and Account.

Constraint:Account type Should be either “**Saving**” or “**Current**”

A.Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B.Execute the following queries in PostGreSQL (any 3) [6 marks]

- Count the number of customers whose name contains substring “in”.
- Give the average balance of all Current accounts.
- Display details of accounts in sorted order of balance.
- Increase the balance of all accounts of customer named as “Sahil” by Rs 10000.
- Add column “phone_no” to customer table.

C.Write a query to list the city-wise customer names having minimum balance. [4 marks]

OR

C. Create a view to list details of customers in Pune having maximum balance in saving account.

Q 1 B)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define SZ 10
```

```
typedef struct stack {
```

```
    int stk[SZ];
```

```
    int top;
```

```
} STACK;
```

```
STACK s1;
```

```
void initStack(STACK *s) {  
    s->top = -1;  
}
```

```
int isFull(STACK *s) {  
    return s->top == SZ - 1;  
}
```

```
int isEmpty(STACK *s) {  
    return s->top == -1;  
}
```

```
void push(STACK *s, int n) {  
    s->stk[++s->top] = n;  
}
```

```
int pop(STACK *s) {  
    return s->stk[s->top--];  
}
```

```
void display(STACK *s) {  
    if (isEmpty(s)) {  
        printf("Stack is empty\n");  
        return;  
    }  
}
```

```

printf("Stack elements (top to bottom): ");
for (int i = s->top; i >= 0; i--) {
    printf("%d ", s->stk[i]);
}
printf("\n");
}

int main() {
    int n, num;

    initStack(&s1);

    printf("Enter number of elements to push: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        if (isFull(&s1)) {
            printf("Stack is full! Cannot push more elements.\n");
            break;
        }
        printf("Enter element %d: ", i + 1);
        scanf("%d", &num);
        push(&s1, num);
    }

    printf("\nAfter pushing elements:\n");
    display(&s1);

    printf("\nPopping elements one by one:\n");

```

```
while (!isEmpty(&s1)) {  
    printf("Popped element: %d\n", pop(&s1));  
    display(&s1);  
}
```

```
return 0;
```

```
}
```

Q 2)

A)

1)

```
SELECT COUNT(*) AS count_customers
```

```
FROM Customer
```

```
WHERE cust_name LIKE '%in%';
```

3)

```
SELECT * FROM Account
```

```
ORDER BY balance;
```

5)

```
ALTER TABLE Customer
```

```
ADD COLUMN phone_no VARCHAR(15);
```

C 2)

```
SELECT c.cust_city, c.cust_name, MIN(a.balance) AS min_balance
```

```
FROM Customer c
```

```
JOIN Account a ON c.cno = a.cno
```

```
GROUP BY c.cust_city, c.cust_name
```

```
ORDER BY c.cust_city;
```

Slip 24]

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q1) Accept the array of n integers from user and sort the array in ascending order by using recursive Counting sort algorithm.

OR

Q1) Perform the following operations using static implementation of Queue

- i. Create a queue of n integers.
- ii. Search the element.
- ii. Delete the element from the queue and display updated queue.

Section II : Database Management Systems

[15 marks]

Q 2) Consider the following database (**primary keys are underlined**).

Bus (Busno , capacity ,depot_name)

Route (Routeno ,source ,destination ,no_of_stations)

Relationship :

Bus-Route : **Many to One**.

Constraint: Bus capacity should not be null.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- i. Count total buses at depot "Sangamner".
- ii. Delete all buses having capacity less than 20.
- iii. List all buses on route number 41.
- iv. Find the maximum number of stations.
- v. List all routes from "Pimpri" to "Vimannagar".

C. Write a query to delete all the buses of destination "Baramati". [4 marks]

OR

C. Create a view to find the route having maximum number of stations and the buses which run on that route.

Q1 B)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 10
```

```
struct queue {
```

```
    int data[MAX];
```

```
    int front, rear;
```

```
};
```

```
void initq(struct queue *q) {  
    q->front = q->rear = -1;  
}
```

```
int isfull(struct queue *q) {  
    return q->rear == MAX - 1;  
}
```

```
int isempty(struct queue *q) {  
    return q->rear == q->front;  
}
```

```
void addq(struct queue *q, int n) {  
    q->rear++;  
    q->data[q->rear] = n;  
}
```

```
int delq(struct queue *q) {  
    q->front++;  
    return q->data[q->front];  
}
```

```
void display(struct queue *q) {  
    if (isempty(q)) {  
        printf("Queue is empty\n");  
        return;  
    }
```

```

printf("Queue elements: ");
for (int i = q->front + 1; i <= q->rear; i++)
    printf("%d ", q->data[i]);
printf("\n");
}

```

```

int search(struct queue *q, int key) {
    for (int i = q->front + 1; i <= q->rear; i++) {
        if (q->data[i] == key)
            return i; // return index
    }
    return -1; // not found
}

```

```

int main() {
    struct queue q;
    initq(&q);
    int n, num, key;

    printf("Enter number of elements to insert (max %d): ", MAX);
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        if (isfull(&q)) {
            printf("Queue is full!\n");
            break;
        }
        printf("Enter element %d: ", i + 1);
        scanf("%d", &num);
    }
}

```

```

        addq(&q, num);
    }

    printf("\nQueue after insertion:\n");
    display(&q);

    printf("\nEnter element to search: ");
    scanf("%d", &key);
    int pos = search(&q, key);
    if (pos != -1)
        printf("Element %d found at position %d in the queue\n", key, pos - q.front);
    else
        printf("Element %d not found in the queue\n", key);

    if (!isempty(&q)) {
        printf("\nDeleted element: %d\n", delq(&q));
        printf("Queue after deletion:\n");
        display(&q);
    } else {
        printf("Queue is empty, nothing to delete.\n");
    }

    return 0;
}

```

Q 2)

A)

CREATE TABLE Route (

 Routeno INT PRIMARY KEY,

```
source VARCHAR(50),  
destination VARCHAR(50),  
no_of_stations INT  
);
```

```
CREATE TABLE Bus (  
    Busno INT PRIMARY KEY,  
    capacity INT NOT NULL,  
    depot_name VARCHAR(50),  
    Routeno INT REFERENCES Route(Routeno)  
);
```

```
INSERT INTO Route VALUES  
(41, 'Pimpri', 'Vimannagar', 12),  
(42, 'Sangamner', 'Baramati', 8),  
(43, 'Pimpri', 'Baramati', 15);
```

```
INSERT INTO Bus VALUES  
(101, 40, 'Sangamner', 41),  
(102, 18, 'Sangamner', 42),  
(103, 35, 'Pimpri', 41),  
(104, 50, 'Pimpri', 43),  
(105, 25, 'Sangamner', 41);
```

B)

2)

```
DELETE FROM Bus
```

```
WHERE capacity < 20;
```

3)

```
SELECT *  
FROM Bus  
WHERE Routeno = 41;
```

4)

```
SELECT MAX(no_of_stations) AS max_stations  
FROM Route;
```

C 1)

```
DELETE FROM Bus  
WHERE Routeno IN (  
    SELECT Routeno FROM Route WHERE destination = 'Baramati'  
);
```

Slip 25]

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 marks]

Q1) b) Write a menu driven program for the following operations on singly linked list.

- Create linked list and display it.
- Accept an element from user and concatenate it with a created link list.

OR

Q 1) Write a menu driven program to perform the following operations on stack

- Accept a postfix expression.
- Evaluation of postfix expression

Section II : Database Management Systems

[15 marks]

Q2) Consider the following database (**Primary keys are underlined**)

Employee (empno, emp_name, city, designation, salary)

Project (projno, proj_name, status, start_date)

There exists a **Many to One** relationship between Employee and Project.

Constraint: status Should be **“Complete”**, **“In progress”**.

A Create the above database in PostGreSQL and insert sufficient records.

[5 marks]

B Execute the following queries in PostGreSQL (any 3)

[6 marks]

- Find the names of all Employees working on project no 105.
- Count the number of Projects which are “In progress”.
- Change start date of project “Robotics” to 15/12/2022.
- Increase the salaries of all employees working on project 10 by 5%.
- Delete records of all completed projects.

C. Write a query to display the project names on which more than 3 employees are working.

[4 marks]

OR

C. Create a view to find all Employees who are working on project which started on ‘1/4/2025’ and status is “in progress”.

Q1 A)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *create();

void display(struct node *head);

struct node *concat_element(struct node *head, int data);

int main() {

    struct node *head = NULL;

    int choice, elem;

    while(1) {

        printf("\n1. Create List\n");
        printf("2. Display List\n");
        printf("3. Concatenate an Element\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {

            case 1:

                head = create();

                break;

            case 2:

                display(head);

                break;

            case 3:

                printf("Enter element to concatenate: ");
                scanf("%d", &elem);

                head = concat_element(head, elem);

                display(head);
```

```

        break;
    case 0:
        exit(0);
    default:
        printf("Invalid choice\n");
    }
}

return 0;
}

// Function to create a linked list
struct node *create() {
    int n, i;

    struct node *head = NULL, *nw, *curr;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        nw = (struct node *)malloc(sizeof(struct node));
        nw->next = NULL;
        printf("Enter data: ");
        scanf("%d", &nw->data);

        if(head == NULL) {
            head = curr = nw;
        } else {
            curr->next = nw;

```

```

        curr = nw;
    }
}

return head;
}

// Function to display linked list
void display(struct node *head) {
    struct node *curr = head;
    if(curr == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Linked List: ");
    while(curr != NULL) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

// Function to concatenate an element at the end of linked list
struct node *concat_element(struct node *head, int data) {
    struct node *nw = (struct node *)malloc(sizeof(struct node));
    nw->data = data;
    nw->next = NULL;

```

```

if(head == NULL) {
    head = nw;
} else {
    struct node *curr = head;
    while(curr->next != NULL) {
        curr = curr->next;
    }
    curr->next = nw;
}

return head;
}

```

Q 2)

A)

```

CREATE TABLE Project (
    projno INT PRIMARY KEY,
    proj_name VARCHAR(50),
    status VARCHAR(20) CHECK (status IN ('Complete','In progress')),
    start_date DATE
);

```

```

CREATE TABLE Employee (
    empno INT PRIMARY KEY,
    emp_name VARCHAR(50),
    city VARCHAR(50),
    designation VARCHAR(50),
    salary NUMERIC(10,2) CHECK (salary > 0),

```

```
projno INT REFERENCES Project(projno) -- Many-to-One relationship
);
```

```
INSERT INTO Project VALUES
(101,'AI Development','Complete','2022-01-10'),
(102,'Web App','In progress','2023-05-20'),
(103,'Mobile App','In progress','2023-07-01'),
(104,'Database Migration','Complete','2022-11-15'),
(105,'Robotics','In progress','2022-12-01');
```

```
INSERT INTO Employee VALUES
(1,'Alice','Pune','Developer',50000,101),
(2,'Bob','Mumbai','Tester',45000,102),
(3,'Charlie','Pune','Developer',55000,103),
(4,'David','Nashik','Manager',70000,105),
(5,'Eve','Pune','Developer',52000,105),
(6,'Frank','Mumbai','Tester',48000,105),
(7,'Grace','Pune','Developer',53000,104);
```

B)

1)

```
SELECT emp_name
FROM Employee
WHERE projno = 105;
```

3)

```
UPDATE Project
SET start_date = '2022-12-15'
WHERE proj_name = 'Robotics';
```

5)

```
DELETE FROM Project  
WHERE status = 'Complete';
```

C 1)

```
SELECT p.proj_name, COUNT(e.empno) AS emp_count  
FROM Project p  
JOIN Employee e ON p.projno = e.projno  
GROUP BY p.proj_name  
HAVING COUNT(e.empno) > 3;
```